WaferMind AI — Project Charter

Version 0.2 · Approved · October 28, 2026 · All documentation and code in English

1. Purpose of this charter

This charter defines what WaferMind AI will build first, how we will judge the results, and what we deliberately leave for later. Once it is approved, work starts with `01_secom_eda.ipynb`. The guiding rule is that every model, metric and architectural component must have a clear purpose.

SECOM supports **historical quality-failure classification**. WaferMind Quality does not claim early prediction unless the timing of the feature measurements can be established (Section 5).

2. Business problem and industrial context

A semiconductor wafer passes through hundreds of process steps. A defect found late wastes the value added by every step after the point where it could have been caught, and inspection and test capacity is limited. Fabs record large volumes of process data, but turning it into early decisions is hard: failures are rare, the signal is spread across hundreds of weakly informative variables, and processes drift over time.

WaferMind AI investigates whether process data can support five practical questions: which units are likely to fail, what defect pattern a wafer shows, whether the process is behaving abnormally, what drives yield, and which equipment is at risk of failure.

This is a proof of method on public datasets. It is not a deployed fab system, and no result should be presented as production-ready.

3. Project scope and use cases

Module	Decision it supports	Techniques	Status
Quality	Which units to prioritize for extra inspection	Classification, feature selection, explainability	Specified (v1)
Vision	Which defect pattern a wafer map shows	CNNs, computer vision, PyTorch	Roadmap

Module	Decision it supports	Techniques	Status
Anomaly	Whether the process drifts from normal	Anomaly detection, statistical methods	Roadmap, reuses SECOM
Yield	What drives yield and how it may move	Statistical modeling, regression, ML	Roadmap, dataset to select
Reliability	Which equipment is at risk of failure	Predictive modeling, time series	Roadmap, dataset to select

In scope for v1: WaferMind Quality, end to end (data understanding, baselines, models, explainability, report).

Out of scope for v1: the other four modules, real-time serving, deployment, dashboards, and any claim of production readiness.

How later modules start: each gets a one-page mini-charter (prediction target, metric, dataset, acceptance criteria) before any modeling begins, and only after the previous phase has passed its exit review.

4. Dataset selection and limitations

Quality dataset: SECOM (UCI Machine Learning Repository). It contains 1,567 production entities with 590 anonymized sensor features, a pass/fail label (1,463 pass, 104 fail, about 6.6% failures) and timestamps covering roughly July to October 2008.

Known limitations, stated up front:

- **Few positives.** With 104 fails, results will have wide uncertainty. A 20% chronological holdout contains only about 20 fails.
- **Anonymized features.** No physical meaning is attached to the 590 variables, so explainability is limited to feature IDs and statistical patterns.
- **Data quality.** Many features have missing values or are constant or near-constant.
- **Narrow context.** One line, one short period, no equipment, recipe or lot information. Results cannot be claimed to generalize to other fabs or processes.
- **Coarse label.** Pass/fail does not say which failure mode occurred.
- **Limited drift analysis.** Timestamps allow a chronological split, but the period is too short to study process drift reliably.

Candidates for later modules (not commitments):

- **Vision:** WM-811K wafer maps. The dataset has 811,457 maps, of which 172,950 are expert-labeled into nine classes (eight defect patterns plus "none"). About 85% of the labeled maps are "none", which leaves 25,519 with a real pattern, and those are strongly imbalanced (Edge-Ring about 38%, Near-Full about 1%).
- **Vision, optional extension:** MixedWM38, with 38,015 maps of 52×52 dies in 38 classes (one normal, eight single patterns, 29 mixed combinations). Part of it is synthetic, generated to balance the classes, so it is used only after WM-811K and reported separately.
- **Anomaly:** the SECOM process features, so no new dataset is needed.
- **Yield and Reliability:** to be selected. Criteria: publicly available, licence permits our use, labels and time structure match the target, and semiconductor-relevant where possible. NASA C-MAPSS is a possible surrogate for Reliability, but it is not semiconductor data and would have to be labeled as such.

What the "images" are. Wafer maps are not photographs. Each one is a grid with one cell per die, produced by electrical testing, with three values: no die, passing die, failing die. A CNN reads the grid as a small image, so compute stays modest. Map sizes differ, so they are resized to a common shape before modeling.

Classification, not detection. The Vision module assigns a defect pattern to a whole wafer. Both datasets provide wafer-level labels only, so locating defects inside a map (segmentation) is out of scope. Detecting defects in optical or SEM photographs would need a different dataset, which has not been selected and is not part of this charter.

Handling and verification. Raw files stay read-only in `data/raw/<dataset>/`, for example `secom/` and `wm811k/`. Common copies of WM-811K are a Python pickle file, which can run code when loaded, so it is loaded only from a trusted copy and converted once to a safe format in `data/processed/`. Licence terms for WM-811K and MixedWM38 are not yet confirmed and must be checked before Phase 3. The figures above come from a published description of both datasets (Giri et al., arXiv 2505.03848); confirm them against the dataset pages at download time.

5. Prediction target (WaferMind Quality)

- **Task framing:** historical quality-failure classification. From the recorded measurements, estimate whether a production entity failed the in-house line test.
- **Target:** binary label, fail = 1, pass = 0 (SECOM codes pass as -1; it is recoded).
- **Unit of prediction:** one production entity as recorded in SECOM.

- **Feature availability (assumption, not verified):** the UCI description gives one row per production entity with its measured features and a pass/fail result, plus a timestamp for the test point. It does not state when each feature was measured relative to the moment a failure could be detected. A model could therefore score well using measurements that arrive too late to act on.
- **Claim boundary and check:** early prediction is not established in v1, and no report will say that the model predicts failures before they occur. Phase 1 starts by checking the dataset documentation and its introductory paper for feature timing and process context, and records the findings in `01_secom_eda.ipynb`. If timing cannot be verified, results stay labeled historical classification.
- **Output:** a fail-risk score. A threshold converts the score into a "flag for inspection" decision.

6. Business KPIs and ML evaluation metrics

Business decision: flag a fixed share of units for extra inspection. In v1 this is evaluated on historical data (Section 5), so it shows how much a risk ranking would have helped, not that the ranking could have been used early.

Business KPIs

- **Inspection budget:** the 10% of units with the highest predicted risk, kept as an experimental assumption until confirmed (Section 10).
- **Fail capture rate at 10%:** the share of all holdout failures that fall among the flagged units.
- **Precision at 10%:** the share of flagged units that are failures. Capture rate says how many failures are found; precision says how much each inspection yields.
- **Random reference:** flagging the same number of units at random captures 10% of failures in expectation, with precision equal to the holdout failure rate. Both are also estimated from 1,000 random draws to show their spread.
- **Cost-weighted outcome:** (cost of a missed fail × missed fails) + (cost of a false alarm × false alarms), compared with no model and with random flagging at the same budget. The cost ratio belongs to the project owner; until it is set, results are reported across 5:1, 10:1 and 20:1.

```
\text{Capture@10\%} = \frac{\text{failures among flagged units}}{\text{all failures in the holdout}}
```

```
\text{Precision@10\%} = \frac{\text{failures among flagged units}}
{\text{number of flagged units}}
```

Selection procedure (fixed before the holdout is scored):

1. Score every unit in the final holdout with the locked model. A higher score means higher risk.
2. Number flagged: k = floor(0.10 × N), where N is the number of holdout units. Rounding down means the budget is never exceeded (for example, N = 313 gives k = 31).
3. Sort by score, highest first. Ties are broken by timestamp (earliest first), then by original row index, so the result is deterministic and uses no label information.
4. Flag the first k units. The budget applies once over the whole holdout; SECOM has no lot or batch identifiers, so working batch by batch is out of scope.
5. Report capture rate and precision at k, each with a bootstrap interval and the number of holdout failures, next to the random reference.

ML metrics

- **Primary:** average precision, compared with the failure prevalence in the holdout (a random ranking is expected to score about that value). The full-dataset prevalence (about 6.6%) is reported separately for context, because a chronological holdout can have a different failure rate.
- **Operating point:** capture rate and precision at the 10% budget, as defined above.
- **Secondary:** ROC-AUC, and Brier score if the output is used as a calibrated risk.
- **Not used as a headline:** accuracy. A model that says "pass" for every unit scores about 93% on the full dataset.
- **Uncertainty:** every reported metric comes with a bootstrap confidence interval and the number of holdout failures. The intervals describe sampling variation only, not temporal change or the effect of model selection.

7. Evaluation rules and model acceptance criteria

Evaluation rules

- **Data roles.** The oldest 80% of units by timestamp form the development set. The newest 20% is the final holdout and is not used for any decision until all choices are locked.

- **Time-aware validation.** Inside the development set, models are compared on expanding-window folds: every validation fold lies later in time than all of its training data, and every model uses the same folds. The number of folds is fixed before modeling and recorded in the experiment log. A timestamp does not guarantee independence between units (units made close together can be correlated), so fold results are read as indicative.
- **Selection inside the pipeline.** Variance filtering of constant and near-constant features, imputation, scaling, feature selection and hyperparameter choice are fitted on training folds only, in one pipeline. The holdout is never used to choose features, thresholds, hyperparameters or models.
- **Lock, then score once.** After the model, features, hyperparameters and the 10% procedure are locked, the final holdout is scored exactly once. Any later change counts as exploratory, and the holdout is reported as already used.
- **Baselines**, on identical folds and the same holdout: (1) prevalence baseline (constant score) and random ranking, whose average precision is estimated over 1,000 random rankings so one lucky or unlucky draw does not decide; (2) regularized logistic regression; (3) one gradient-boosted tree model. A more complex model is adopted only if it improves on the previous step by more than the uncertainty seen across validation folds.
- **Limits of the holdout.** The newest 20% holds about 20 failures, covers a short span, and may have a different failure rate from the development period. It is a rough check, not a precise estimate, and cannot show how the model would behave in a later period or on another line.
- **Reading explanations.** Feature importance and SHAP values describe associations the model learned from historical data. They do not show that a sensor causes a failure.

Acceptance assessment (proposed, to be confirmed). The result is judged on the evidence below and ends in one of three outcomes. No single test decides it, and thresholds are not changed after results are seen.

1. **Integrity (required):** automated tests confirm that the split is chronological and that nothing was fitted or chosen using holdout data. If this fails, the result is invalid.
2. **Average precision:** holdout average precision compared with the holdout failure prevalence, with a bootstrap interval and the number of holdout failures.
3. **Budget metrics:** capture rate and precision at the 10% budget, compared with random selection at the same budget (expected value and spread over random draws), each with an interval.

4. **Consistency:** the time-aware validation folds and the holdout point in the same direction.
5. **Limitations:** the points in Sections 4 and 5, and the short holdout, are stated next to the results.

Possible outcomes

- **Supported:** integrity passes, holdout average precision and capture rate exceed the random reference with intervals that clear it, and the validation folds agree.
- **Not supported:** integrity passes, but the holdout estimates are at or below the random reference, or a validation gain disappears on the holdout.
- **Inconclusive:** estimates are above the reference but the intervals overlap it, or the holdout has too few failures to judge. The report says so and does not tune the model to change the outcome.

A supported outcome means evidence of a usable signal for historical failure classification. It does not establish early prediction (Section 5) or production readiness. The intervals describe sampling variation only and do not capture temporal change or the effect of model selection, so conclusions are stated with that caution. A not-supported or inconclusive result is a legitimate outcome and is reported as such.

8. Initial architecture

The principle is the simplest setup that supports reproducible notebooks and tested code.

```
wafermind-ai/
├── README.md
├── pyproject.toml
├── .gitignore
├── docs/
│   ├── project-charter.md
│   ├── experiment-log.md          data record, run log
│   └── figures/
├── data/
│   ├── raw/                       read-only, not in Git
│   └── processed/                 not in Git
├── notebooks/
│   ├── 01_secom_eda.ipynb
│   ├── 02_secom_baselines.ipynb
│   ├── 03_secom_feature_selection_and_trees.ipynb
```

```
|   └── 04_secom_explainability_and_holdout.ipynb
├── src/wafermind/
|   ├── __init__.py
|   ├── ingestion.py
|   ├── splits.py                  chronological split, time-aware folds
|   ├── preprocessing.py           variance filter, imputation, selection
|   ├── train.py
|   └── metrics.py                 AP, capture rate, precision at budget
├── tests/
|   ├── test_ingestion.py
|   ├── test_splits.py             leakage checks
|   ├── test_preprocessing.py
|   └── test_metrics.py            includes tie-breaking
└── artifacts/                     model files, run outputs, not in Git
```

- **Stack for v1:** Python, pandas, scikit-learn, matplotlib and seaborn, SHAP for explainability, pytest. Gradient boosting through scikit-learn or LightGBM. Dependencies are pinned in `pyproject.toml`.
- **Not used in v1:** PyTorch, Databricks, dbt and Docker. PyTorch is introduced when the Vision module starts. dbt would be revisited if a later module needs several tables with tracked transformations, Databricks if data volume outgrows a laptop, and Docker only if the pinned Python environment cannot reproduce results across machines.
- **Notebooks and source code:** notebooks are for exploration and reporting. Stable reusable logic moves into `src/wafermind/` when it is shared across notebooks or needs automated tests; it does not have to be duplicated first. The chronological split, the preprocessing pipeline and the budget metrics are tested code from the start.
- **Reproducibility record:** `docs/experiment-log.md` is committed to Git. It holds the dataset source and version, the licence verification and file checksums, the dependency versions and random seeds, and for each experiment its configuration, metrics and the Git commit of the code. The selected model's configuration is recorded the same way. There is no tracking server.
- **Kept out of Git:** raw and processed data and trained model files (`data/`, `artifacts/`), unless a specific, documented reason justifies versioning one. The log's source and checksum let anyone re-create them.

9. Development roadmap

Phase	Content	Exit criterion
0. Charter	Review and approve this document	Section 10 decisions settled
1. Quality	The four SECOM notebooks in Section 8; `01` starts with the feature-timing check	Acceptance assessment done, report written
2. Anomaly	Mini-charter; SECOM features; statistical baseline first	Mini-charter criteria evaluated
3. Vision	Mini-charter; WM-811K; simple CNN in PyTorch	Mini-charter criteria evaluated
4. Yield and Reliability	Dataset selection with an explicit go/no-go	Datasets chosen, or module descoped with the reason documented

Phases run in order and each starts only after the previous exit review. No calendar dates are set until Phase 1 effort has been measured.

10. Decisions (approved)

1. **Inspection budget:** 10%, kept as an experimental assumption.
2. **Cost ratio:** results are reported across 5:1, 10:1 and 20:1.
3. **Selection rules:** the flagged count is rounded down (floor of 10% of holdout units); ties are broken by timestamp, then original row index.
4. **Holdout size:** 80/20 chronological split, accepting about 20 holdout failures and a plausible inconclusive outcome.

Settled in v0.2: Quality-only v1 scope; historical failure classification with no early-prediction claim until feature timing is established; a three-outcome acceptance assessment; no PyTorch, Databricks, dbt or Docker in v1.